

untangle::executor

Generated by Doxygen 1.18.0

1 executor	1
2 Namespace Index	5
2.1 Namespace List	5
3 Class Index	7
3.1 Class List	7
4 File Index	9
4.1 File List	9
5 Namespace Documentation	11
5.1 untangle Namespace Reference	11
5.1.1 Detailed Description	11
6 Class Documentation	13
6.1 untangle::executor< taskT > Class Template Reference	13
6.1.1 Detailed Description	13
6.1.2 Constructor & Destructor Documentation	14
6.1.2.1 executor()	14
6.1.2.2 ~executor()	14
6.1.3 Member Function Documentation	15
6.1.3.1 add_task()	15
6.1.3.2 start()	15
6.1.3.3 stop()	15
6.1.4 Member Data Documentation	16
6.1.4.1 on_task_error	16
7 File Documentation	17
7.1 executor.hpp	17
Index	21

Chapter 1

executor

c++ thread pool executor based on async execution

The interface is one header that exposes a pool of workers, each one an **async** execution in continuous mode, fed by a shared queue of tasks.

This is the prototype, imported. It was written in `async/prototypes/` to answer whether execution is a usable building block for a pool — it is — and it is being hardened here. The behaviour below is what the tests state today, not a settled interface.

requirements

C++23, and the `async` submodule:

```
git submodule update --init --recursive
```

`async` carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with `std::println`, so they need a standard library that provides `<print>` — GCC 14 or newer.

example

```
#include <executor.hpp>
#include <functional>

int main() {
    untangle::executor<std::function<void(void)>> pool(4);
    pool.start();

    for (int i = 0; i < 100; ++i) {
        pool.add_task([i] { work(i); });
    }

    return 0; // ~executor finishes what was added, then stops the workers
}
```

the task type

The pool is a template on the task it runs, the way an execution is a template on its action. One pool runs one signature, and the caller names it:

```
untangle::executor<std::function<int(void)>> pool(4); // returns are run and dropped
```

The task type does not have to be a `std::function`. What an execution requires of it is a nested `result_<_type`, which it reads rather than deduces, so any callable declaring one will do — and a pool built on such a callable never touches `std::function::result_type`, which the standard removed in C++20 and both `libc++` and `libstdc++` still supply by grace rather than contract.

```
struct my_task {
    using result_type = void;
    void operator()() const;
};
```

```
untangle::executor<my_task> pool(4);
```

A task type that takes arguments is refused, with a `static_assert` rather than an error inside `std::bind`: the pool queues tasks, and has nowhere to keep arguments for them until a worker frees up.

how work is placed

N executions run in continuous mode behind a shared `std::deque` of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue finished — on `_finished`, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. `execution::is_busy()` is what `add_task()` reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

running, stopping, restarting

A pool is built stopped. The constructor makes the workers and wires them up; `start()` runs them. Until it is called the pool refuses work, because a task taken then would sit unrun and the destructor would wait for it. It is also the moment to assign `on_task_error`, before anything can throw.

`stop()` stops the workers, and stops the pool accepting. What a worker is already running it finishes; what is still queued stays queued. `stop()` does not wait for the workers to leave their threads — only the destructor does that, and that wait is what makes destroying the pool safe.

`start()` after `stop()` restarts it, workers and all, and the queue it was holding drains as they pick it up again. Either call is harmless twice over.

lifetime and failure

The destructor finishes what was added. It stops accepting, waits for the queue to empty and for every worker to go idle, and only then stops them. A task added before the pool goes out of scope has run by the time it does.

A pool destroyed while stopped is the exception: its queue can never empty, because only a running worker takes from it, so the destructor does not wait for it. Whatever is left is reported on stderr and dropped.

`add_task()` says whether the task was taken. It returns false before `start()`, after `stop()`, and once the destructor has begun. It returns false again if the worker it was offered to has stopped. A refused task is destroyed rather than queued, so a false answer means it will not run.

A task that is still running is refused too, and that is chosen rather than incidental. A pool shutting down is waiting for exactly that task, so it is tempting to let the work it spawns through — but then a task that re-adds itself could keep the shutdown from ever ending. One rule instead: a pool that is not running takes no work, whoever is asking. `add_task()` answers false inside the task, so it can tell.

A task that throws does not take the worker with it, and what it threw is not lost. The worker catches it and carries on with the next task; `on_task_error` is where the throw goes:

```
untangle::executor<std::function<void(void)>> pool(4);

pool.on_task_error = [](std::exception_ptr thrown) {
    try {
        std::rethrow_exception(thrown);
    } catch (const std::exception& e) {
        log(e.what());
    }
};
```

It carries a `std::exception_ptr` because a task may throw something that is not a `std::exception`, and that is the case most worth hearing about. Assign it before the first `add_task()` — a worker reads it — and expect it on a worker's thread, with more than one worker possibly inside it at once. Leave it unset and the pool warns on stderr instead, naming the worker.

A return value is still not collected: a continuous worker keeps none, so a task with something to report carries its own channel.

building the tests

```
cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build
```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access.

sanitizers

`EXECUTOR_SANITIZE` builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are `address`, `thread`, `undefined`, or `empty`. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. `address` and `thread` instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

tools/make_doc.sh builds doc/refman.pdf from the header and this README. It needs doxygen, tectonic and python3, and it fails on any doxygen warning that is not an obsolete-tag notice, so an undocumented member or a broken \ref does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	A thread pool executor built out of <code>untangle::async::execution</code> objects	11
--------------------------	---	--------------------

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::executor< taskT >	
Runs tasks on a fixed pool of untangle::async::execution workers	13

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

executor.hpp	17
--	----

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool executor built out of `untangle::async::execution` objects.

Classes

- class `executor`
Runs tasks on a fixed pool of `untangle::async::execution` workers.

5.1.1 Detailed Description

A thread pool executor built out of `untangle::async::execution` objects.

N workers run in continuous mode behind a shared queue. A task goes straight to a free worker only when the queue is empty; otherwise it joins the back, and a worker takes the front as it frees up.

Chapter 6

Class Documentation

6.1 `untangle::executor< taskT >` Class Template Reference

Runs tasks on a fixed pool of `untangle::async::execution` workers.

```
#include <executor.hpp>
```

Public Member Functions

- `executor` (`std::size_t worker_count`)
Builds `worker_count` workers. They do not run until `start()`.
- `~executor` ()
Finishes what has been added, then stops every worker.
- `bool add_task` (`taskT task`)
Queues a task, or hands it to a worker that has nothing to do.
- `std::size_t pending` () `const`
How many tasks are waiting for a worker.
- `void start` ()
Starts every worker, and starts the pool accepting work.
- `void stop` ()
Stops every worker. What they are already running, they finish.

Public Attributes

- `std::function< void(std::exception_ptr)> on_task_error`
Called with whatever a task threw. Assigned by the caller.

6.1.1 Detailed Description

```
template<typename taskT>  
class untangle::executor< taskT >
```

Runs tasks on a fixed pool of `untangle::async::execution` workers.

Template Parameters

taskT	The type of the task the pool runs. One pool runs one signature, and a non-void return is run and dropped.
-------	--

6.1.2 Constructor & Destructor Documentation

6.1.2.1 executor()

```
template<typename taskT>
untangle::executor< taskT >::executor (
    std::size_t worker_count) [inline], [explicit]
```

Builds worker_count workers. They do not run until [start\(\)](#).

Parameters

worker_count	- How many workers to run. Must not be zero.
--------------	--

Exceptions

std::invalid_argument	- worker_count is zero. Such a pool would take work it could never run.
-----------------------	---

6.1.2.2 ~executor()

```
template<typename taskT>
untangle::executor< taskT >::~executor () [inline]
```

Finishes what has been added, then stops every worker.

Both waits are unbounded, and both report on stderr while they last, naming the workers they are waiting on.

Attention

A task still running here is refused if it adds more work, even though this is waiting for that very task. It is deliberate: a shutdown that took new work could be kept from ever ending by a task that re-adds itself. [add_task\(\)](#) answers false, so the task can see it.

6.1.3 Member Function Documentation

6.1.3.1 add_task()

```
template<typename taskT>
bool untangle::executor< taskT >::add_task (
    taskT task) [inline]
```

Queues a task, or hands it to a worker that has nothing to do.

Remarks

The first free worker takes it, counting from the start, so a pool that is not busy keeps giving work to the same one. That is the intent: the worker that just ran a task is the warm one, and a free worker is free whichever it is.

Returns

true - the task was accepted. false - it was refused, by a pool that has not been started, has been stopped, or is being destroyed, or by a worker that has stopped. Either way it will not run. A task calling this from a worker thread is answered like any other caller.

6.1.3.2 start()

```
template<typename taskT>
void untangle::executor< taskT >::start () [inline]
```

Starts every worker, and starts the pool accepting work.

A pool refuses work until this is called, and refuses it again after [stop\(\)](#) until this is called a second time - a stopped pool is restarted by it, workers and all. Calling it twice over is harmless.

6.1.3.3 stop()

```
template<typename taskT>
void untangle::executor< taskT >::stop () [inline]
```

Stops every worker. What they are already running, they finish.

Queued tasks stay unrun and a task added afterwards is refused. That holds whoever is asking, a task already running on a worker included - a pool that is not running takes no work. [start\(\)](#) gives the workers a new working life. Calling it twice is harmless.

Attention

It does not wait for the workers to leave their threads. Only the destructor does that, and that wait is what makes destroying the pool safe.

6.1.4 Member Data Documentation

6.1.4.1 `on_task_error`

```
template<typename taskT>  
std::function<void(std::exception_ptr)> untangle::executor< taskT >::on_task_error
```

Called with whatever a task threw. Assigned by the caller.

Nothing else reports a failed task: [add_task\(\)](#) has answered long before the task runs. Left unset, the pool warns on stderr instead.

Remarks

It receives a `std::exception_ptr`, because a task may throw what is not a `std::exception`. Rethrow it to read it.

Attention

It runs on a worker's thread, and several workers may be in it at once. Assign it before the first [add_task\(\)](#), and let nothing escape it.

The documentation for this class was generated from the following file:

- `executor.hpp`

Chapter 7

File Documentation

7.1 executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00010 #pragma once
00011
00012 #include <algorithm>
00013 #include <async.hpp>
00014 #include <atomic>
00015 #include <chrono>
00016 #include <condition_variable>
00017 #include <cstdint>
00018 #include <deque>
00019 #include <exception>
00020 #include <functional>
00021 #include <memory>
00022 #include <mutex>
00023 #include <print>
00024 #include <stdexcept>
00025 #include <string>
00026 #include <thread>
00027 #include <type_traits>
00028 #include <utility>
00029 #include <vector>
00030
00031 namespace untangle {
00032
00039 template <typename taskT>
00040 class executor {
00041     // A task is queued as it stands, with nothing bound to it, so it must take no arguments.
00042     static_assert(std::is_invocable_v<taskT>,
00043         "executor: the task type must be callable with no arguments");
00044
00045 public:
00054     explicit executor(std::size_t worker_count) {
00055         if (worker_count == 0) {
00056             throw std::invalid_argument("executor: worker_count must not be zero");
00057         }
00058
00059         workers_.reserve(worker_count);
00060
00061         for (std::size_t index = 0; index < worker_count; ++index) {
00062             auto next = worker_execution::create_instance("pool_worker_" + std::to_string(index));
00063
00064             // Tells the pool the moment this worker has room for more.
00065             next->on_finished = [this, index] { take_next_task(index); };
00066
00067             // Carries what a task throws to the caller. Assigned here because a worker thread reads it.
00068             next->on_error = [this, name = next->name](std::exception_ptr thrown) {
00069                 report_task_error(name, thrown);
00070             };
00071
00072             // Adds the worker to the poll the destructor waits on.
00073             poll_.add(*next);
00074
00075             workers_.push_back(std::move(next));
00076         }
00077     }
00078 }
```

```

00090 ~executor() {
00091     // Whether the pool was still running when it was destroyed. A stopped one cannot empty its
00092     // queue - only a worker takes from it, through on_finished, and a stopped worker raises no more
00093     // - so its queue is not waited for.
00094     const bool was_running = running_.exchange(false);
00095
00096     {
00097         std::unique_lock<std::mutex> lock(mutex_);
00098
00099         // Waits for the queue to empty and every worker to go idle.
00100         auto interval = std::chrono::milliseconds(report_first_ms);
00101         auto waited = std::chrono::milliseconds(0);
00102
00103         while (!finished_cv_.wait_for(lock, interval, [this, was_running] {
00104             return nothing_running() && (pending_.empty() || !was_running);
00105         })) {
00106             waited += interval;
00107
00108             // Under the lock, so the counts agree with the predicate that just failed.
00109             std::println(
00110                 stderr, "executor: still waiting to finish after {}s - {} queued, {} in a task{}",
00111                 waited.count() / 1000, pending_.size(), count_workers(&worker_execution::is_busy),
00112                 name_workers(&worker_execution::is_busy));
00113
00114             interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00115         }
00116
00117         if (!pending_.empty()) {
00118             std::println(stderr,
00119                 "executor: {} queued task(s) dropped, the pool was stopped before they ran",
00120                 pending_.size());
00121         }
00122     }
00123
00124     stop();
00125
00126     // Waits for every worker to leave its thread, which is what makes destroying them safe. A
00127     // different question from the wait above: a worker can be idle and still be in its thread.
00128     auto interval = std::chrono::milliseconds(report_first_ms);
00129     auto waited = std::chrono::milliseconds(0);
00130
00131     while (poll_.is_running()) { // time of check
00132         const auto tick = std::chrono::milliseconds(poll_interval_ms);
00133         std::this_thread::sleep_for(tick);
00134         waited += tick;
00135
00136         if (waited >= interval) {
00137             std::println(stderr, "executor: still waiting to stop after {}s - {} not left its thread{}",
00138                 waited.count() / 1000, count_workers(&worker_execution::is_running),
00139                 name_workers(&worker_execution::is_running));
00140
00141             interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00142         }
00143     }
00144
00145     // time of use: no worker thread is left to reach workers_.
00146     workers_.clear();
00147 }
00148
00149 bool add_task(taskT task) {
00150     if (!running_) {
00151         return false;
00152     }
00153
00154     std::lock_guard<std::mutex> lock(mutex_);
00155
00156     // The queue comes first: a task does not overtake one already waiting in it.
00157     if (pending_.empty()) {
00158         // Always from the start, so a free pool reuses its warmest worker rather than waking a cold
00159         // one. Under load the workers are busy and the queue below is what spreads the work.
00160         for (std::size_t index = 0; index < workers_.size(); ++index) {
00161             if (!workers_[index]->is_busy()) {
00162                 // A refused task is already destroyed, and stop() stops every worker together, so there
00163                 // is nothing to queue and no other worker to try.
00164                 return give_to_worker(index, std::move(task));
00165             }
00166         }
00167     }
00168
00169     pending_.push_back(std::move(task));
00170     return true;
00171 }
00172
00173 std::size_t pending() const {
00174     std::lock_guard<std::mutex> lock(mutex_);
00175     return pending_.size();
00176 }

```

```

00191
00192 void start() {
00193     // The workers first, so anything accepted after this is handed to one that is running.
00194     for (auto& one : workers_) {
00195         one->start();
00196     }
00197     running_ = true;
00198 }
00199
00200 void stop() {
00201     // First, so that nothing is accepted for workers that are about to stop. A task already on a
00202     // worker still runs; one arriving from here is refused.
00203     running_ = false;
00204
00205     // No lock: a worker waking from stop() runs on_finished, which takes mutex_.
00206     for (auto& one : workers_) {
00207         one->stop();
00208     }
00209 }
00210
00211 std::function<void(std::exception_ptr)> on_task_error;
00212
00213 private:
00214 void report_task_error(const std::string& worker, std::exception_ptr thrown) {
00215     if (on_task_error) {
00216         on_task_error(thrown);
00217         return;
00218     }
00219     try {
00220         std::rethrow_exception(thrown);
00221     } catch (const std::exception& e) {
00222         std::println(stderr, "warning: executor task on '{}' threw: {}", worker, e.what());
00223     } catch (...) {
00224         std::println(stderr, "warning: executor task on '{}' threw an unknown type", worker);
00225     }
00226 }
00227
00228 using worker_execution = untangle::async::execution<taskT>;
00229
00230 [[nodiscard]] bool give_to_worker(std::size_t index, taskT task) {
00231     // add_action() takes the worker's action_mutex under mutex_. The two never cycle, because a
00232     // worker calls on_finished with its action_mutex released.
00233     return workers_[index]->add_action(std::move(task));
00234 }
00235
00236 void take_next_task(std::size_t index) {
00237     std::lock_guard<std::mutex> lock(mutex_);
00238
00239     if (!pending_.empty()) {
00240         taskT next = std::move(pending_.front());
00241         pending_.pop_front();
00242
00243         if (!give_to_worker(index, std::move(next))) {
00244             // The task is already destroyed, so it is reported rather than put back.
00245             std::println(stderr, "executor: {} refused a queued task, which is lost",
00246                 workers_[index]->name);
00247         }
00248     }
00249     return;
00250 }
00251
00252 if (nothing_running()) {
00253     finished_cv_.notify_all();
00254 }
00255 }
00256
00257 bool nothing_running() const {
00258     for (const auto& one : workers_) {
00259         if (one->is_busy()) {
00260             return false;
00261         }
00262     }
00263     return true;
00264 }
00265
00266 std::size_t count_workers(bool (worker_execution::*state)() const) const {
00267     std::size_t count = 0;
00268     for (const auto& one : workers_) {
00269         if ((one.get()->*state)()) {
00270             ++count;
00271         }
00272     }
00273 }
00274
00275

```

```
00332     return count;
00333 }
00334
00341 std::string name_workers(bool (worker_execution::*state)() const) const {
00342     std::string names;
00343
00344     for (const auto& one : workers_) {
00345         if ((one.get()->*state)()) {
00346             names += names.empty() ? "": " : ", ";
00347             names += one->name;
00348         }
00349     }
00350
00351     return names;
00352 }
00353
00355 static constexpr int poll_interval_ms = 50;
00356
00358 static constexpr int report_first_ms = 1000;
00359 static constexpr int report_max_ms = 30000;
00360
00361 mutable std::mutex mutex_;
00362 std::condition_variable finished_cv_;
00363
00369 async::execution_poll poll_;
00370
00371 std::deque<taskT> pending_;
00372 std::vector<std::shared_ptr<worker_execution> > workers_;
00374 std::atomic_bool running_ = {false};
00375 };
00376
00377 } // namespace untangle
```

Index

- `~executor`
 - `untangle::executor< taskT >`, [14](#)
- `add_task`
 - `untangle::executor< taskT >`, [15](#)
- `executor`, [1](#)
 - `untangle::executor< taskT >`, [14](#)
- `on_task_error`
 - `untangle::executor< taskT >`, [16](#)
- `start`
 - `untangle::executor< taskT >`, [15](#)
- `stop`
 - `untangle::executor< taskT >`, [15](#)
- `untangle`, [11](#)
- `untangle::executor< taskT >`, [13](#)
 - `~executor`, [14](#)
 - `add_task`, [15](#)
 - `executor`, [14](#)
 - `on_task_error`, [16](#)
 - `start`, [15](#)
 - `stop`, [15](#)